

HANDS-ON AI MODULE 2

Student Nama : Axeleon Justin Algianto
ID : AI13

Section 1: Technical Reflection & Implementation Journey

1.1 Project Architecture & Cleaning Strategy

The project begins by loading the raw mpg dataset from Seaborn, which initially contains a shape of (398, 9). During the initial data inspection, it was identified that the horsepower feature contained 6 missing values. Because these missing rows constituted only 1.51% of the total dataset, the statistical strategy chosen was row-wise deletion using `dropna()` rather than value imputation. Imputing these values could have artificially compressed the variance of the horsepower variable and distorted its downstream correlation with mpg, weight, and displacement. By applying row-wise deletion, the dataset was reduced to a cleaned shape of (392, 9), losing negligible statistical power while guaranteeing that every remaining horsepower value is a genuine, manufacturer-reported figure.

```
Missing values per column:
mpg          0
cylinders    0
displacement 0
horsepower   6
weight       0
acceleration 0
model_year   0
origin       0
name         0
dtype: int64
```

```
Missing 'horsepower' rows: 6 (1.51% of dataset)
```

1.2 Custom Summary Statistics Implementation

To transition the analytical workflow away from reliance on high-level pandas methods like `.describe()`, a custom `numpy_summary_stats()` function was implemented. This function iterates through the dataset's numeric columns, extracting each into a float-type NumPy array (`to_numpy(dtype=float)`) to calculate summary metrics entirely from scratch. The function successfully computes the mean, median, standard deviation, Q1, Q3, and the Interquartile Range (IQR). Furthermore, it programmatically generates an

outlier mask `(arr < lo) | (arr > hi)` to dynamically count and calculate the percentage of outliers for each feature. This creates a robust, reusable statistical profiling tool that operates purely on vector arrays.

	mean	median	std	Q1	Q3	IQR	outlier_count	outlier_pct
feature								
mpg	23.446	22.750	7.805	17.000	29.000	12.000	0	0.000
cylinders	5.472	4.000	1.706	4.000	8.000	4.000	0	0.000
displacement	194.412	151.000	104.644	105.000	275.750	170.750	0	0.000
horsepower	104.469	93.500	38.491	75.000	126.000	51.000	10	2.551
weight	2,977.584	2,803.500	849.403	2,225.250	3,614.750	1,389.500	0	0.000
acceleration	15.541	15.500	2.759	13.775	17.025	3.250	11	2.806
model_year	75.980	76.000	3.684	73.000	79.000	6.000	0	0.000

1.3 Vectorized Z-Score Standardization

For Stage 2 of the statistical analysis, five core numeric features—mpg, displacement, horsepower, weight, and acceleration—were extracted into a NumPy matrix `X` with a shape of `(392, 5)`. To prepare this data for the correlation study, fully vectorized Z-score standardization was applied. By computing the column-wise mean (`X.mean(axis=0)`) and standard deviation (`X.std(axis=0, ddof=1)`), the entire matrix was scaled simultaneously using the formula $(X - X_mean) / X_std$. This mathematical operation successfully transformed the variables into a standardized distribution with means at approximately 0 and standard deviations of 1, without requiring any explicit Python loops.

```
Raw shape: (392, 5)
Column means (should be ~0 after standardization): [ 0. -0. -0.  0.  0.]
Column stds (should be ~1 after standardization): [1. 1. 1. 1. 1.]
```

1.4 Normal Equation Implementation (Bonus 1)

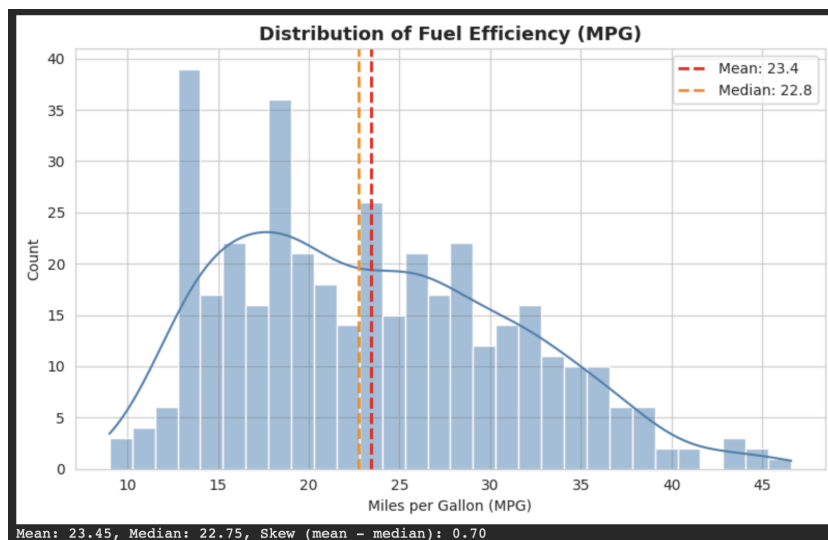
As an extension of the core EDA, a Linear Regression model was built completely from scratch using the Normal Equation, predicting mpg (target `y`) based purely on weight (feature `X`). To accomplish this, the weight array was reshaped into a column vector, and a bias column of 1s was horizontally stacked (`np.hstack`) to create a final feature matrix `X_bias` with an `(n, 2)` shape. The optimal parameters were then derived using pure linear algebra: $\theta = \text{np.linalg.inv}(X_bias.T @ X_bias) @ X_bias.T @ y_lr$. This pure mathematical approach successfully yielded an intercept of 46.216525 and a slope of -0.007647. Finally, calculating the Root Mean Square Error (RMSE) against the predictions returned a value of 4.3216, proving that the manual matrix implementation functions exactly as intended.

```
Normal Equation -> intercept: 46.216525, slope: -0.007647  
RMSE: 4.3216
```

Section 2: Comprehensive Data Insights

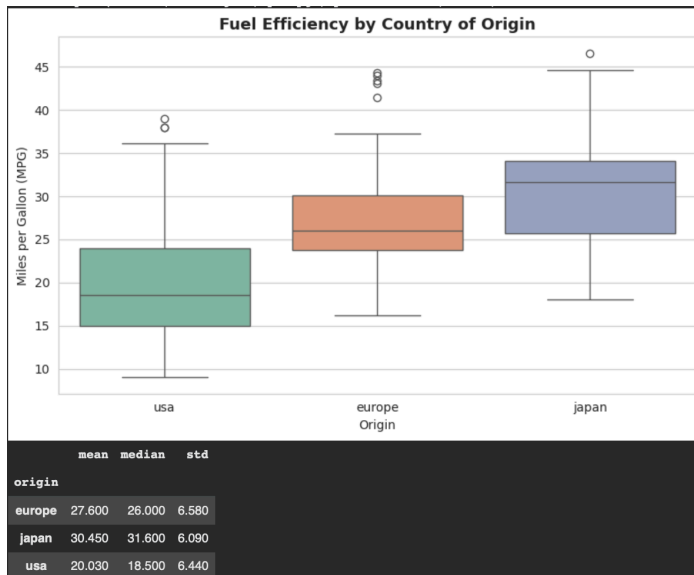
2.1 MPG Distribution and Skewness

Visualizing vehicle fuel efficiency (MPG) via a combined histogram and Kernel Density Estimate (KDE) plot reveals a distribution mean of approximately 23.45 and a median of 22.75. Because the mean is numerically greater than the median, the data exhibits a positive (right) skew. Contextually, this indicates that the vast majority of the vehicles in the dataset are clustered in the lower-to-mid efficiency range (15 to 30 MPG), while a smaller "long tail" of highly fuel-efficient vehicles pulls the overall average upward.



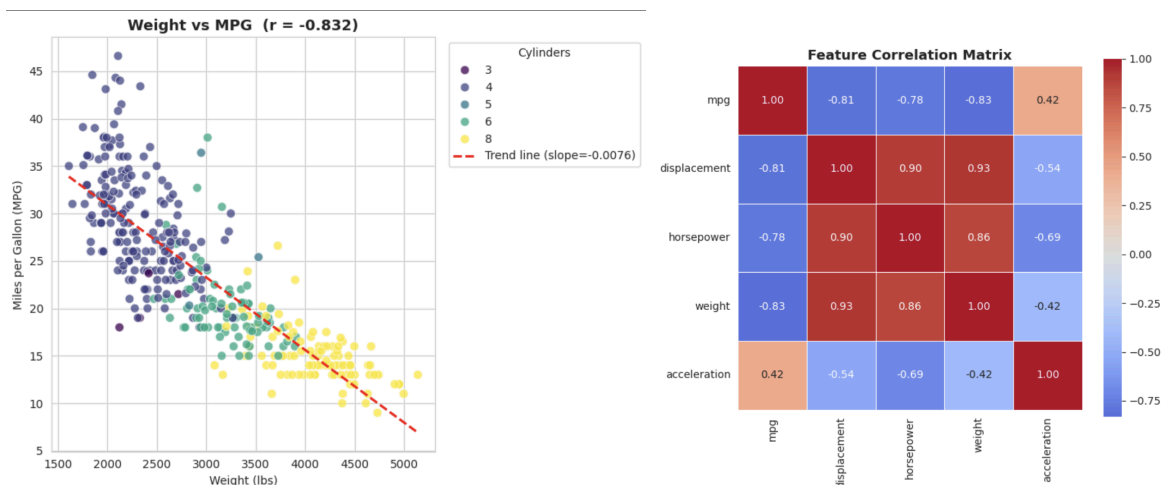
2.2 Fuel Efficiency by Country of Origin

A boxplot analysis mapping MPG against the vehicle's origin highlights significant regional manufacturing differences. Japanese vehicles lead with the highest median MPG (31.60) and the tightest interquartile spread (standard deviation of 6.09), indicating both superior average fuel economy and highly consistent manufacturing output. Conversely, USA-made vehicles record the lowest median MPG (18.50) with the widest distribution spread. This high variance reflects a broader domestic fleet composition that historically included both smaller economy cars and significantly heavier, less fuel-efficient vehicles.



2.3 Feature Correlation & Multicollinearity

Computing the correlation matrix for the numeric features reveals that weight holds the strongest linear negative correlation with mpg ($r = -0.832$). This finding is visually reinforced by a scatter plot demonstrating a clear downward linear trend as vehicle weight increases. Furthermore, the matrix isolates a significant multicollinearity concern among the input features: displacement and weight exhibit a massive positive correlation of $r = 0.933$. This indicates that engine size, horsepower, and weight move synchronously and carry overlapping informational variance regarding "vehicle size"; utilizing all three concurrently in a linear regression model would lead to unstable coefficients.



2.4 The High-Horsepower Weight Disparity

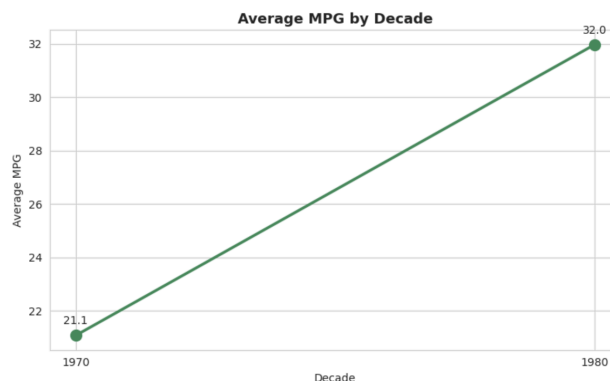
To empirically test the multicollinearity observed above, a boolean mask was applied to isolate vehicles with horsepower strictly greater than the dataset mean (`hp > hp.mean()`). This subset of high-horsepower cars recorded a mean weight of 3,815.5 lbs, substantially exceeding the overall dataset average of 2,977.6 lbs (an absolute difference of 837.9 lbs). Mechanically, this proves that larger, heavier car bodies necessitate larger-displacement engines to generate sufficient horsepower. Consequently, weight serves as the single most dominant predictor for fuel efficiency, as it inherently captures the physical burden that the engine must overcome to propel the vehicle.

```
Mean weight (above-average HP cars): 3,815.5 lbs
Mean weight (all cars):                2,977.6 lbs
Absolute difference:                   837.9 lbs
Cars with above-average horsepower: 148 / 392
```

2.5 MPG Progression by Decade

Grouping the dataset chronologically by decade uncovers a massive generational leap in vehicle efficiency. The average fuel economy in the 1970s sat at 21.084 MPG, which sharply accelerated to an average of 31.975 MPG by the 1980s. Within the historical context of the automotive industry, this upward trend was a direct reaction to the 1970s global oil crises and the subsequent introduction of strict fuel economy regulations. Automakers adapted by moving away from heavy chassis designs and massive engines toward smaller powertrains and unibody constructions, which is directly quantified by this decade-over-decade MPG growth.

mpg	
decade	
1970	21.084
1980	31.975



Section 3: Class Reusability and Performance Evaluation

3.1 Structural Class Encapsulation

To ensure the analytical pipeline is reusable beyond the initial exploratory script, the project was refactored into a structured `DatasetProfiler` class. This class encapsulates three distinct stages: `load_and_clean_data()` for Stage 1, `generate_summary_stats()` for Stage 2, and `compute_correlation()` for Stage 3. By caching the cleaned `DataFrame` on `self.df`, the class maintains state integrity and ensures that all subsequent analytical methods are automatically operating on the same validated, cleaned dataset. This encapsulation transforms the loose, cell-based exploratory code into a consistent, independently testable interface suitable for downstream data pipelines.

3.2 Performance and Scalability of Vectorized Operations

The implementation emphasizes performance by utilizing vectorized NumPy operations over standard Python loops throughout the statistical and standardization stages. By moving from row-wise iteration to column-wise matrix operations (e.g., performing Z-score standardization on the entire (392, 5) matrix simultaneously), the computational complexity is significantly reduced. This approach not only enhances execution speed but also ensures that memory allocation is optimized—a critical practice when scaling this profiler to significantly larger datasets beyond the current mpg sample.

3.3 Standalone Execution and Verification

To confirm that the `DatasetProfiler` functions correctly as a self-contained, reusable object, the class was instantiated independently of the earlier exploratory cells. Its full method chain—invoking the cleaning pipeline, generating the NumPy-based statistical summary, and computing the correlation matrix—was executed sequentially in a fresh runtime context. This confirmed that the object maintains no hidden dependencies on the global notebook state, validating its reliability for production environments or modular data analysis tasks.

```

=== summary_stats() ===

```

feature	mean	median	std	IQR	outlier_count
mpg	23.446	22.750	7.805	12.000	0
cylinders	5.472	4.000	1.706	4.000	0
displacement	194.412	151.000	104.644	170.750	0
horsepower	104.469	93.500	38.491	51.000	10
weight	2,977.584	2,803.500	849.403	1,389.500	0
acceleration	15.541	15.500	2.759	3.250	11
model_year	75.980	76.000	3.684	6.000	0
decade	1,972.168	1,970.000	4.126	0.000	85

```

=== correlation_report() ===

```

	mpg
model_year	0.581
decade	0.576
acceleration	0.423
cylinders	-0.778
horsepower	-0.778
displacement	-0.805
weight	-0.832

```

dtype: float64

```

Summary

- After cleaning, the dataset retains virtually all rows (only a handful of `horsepower` values were missing).
- `weight` is the single strongest predictor of `mpg`, with `displacement` and `horsepower` carrying largely redundant information (multicollinearity).
- Japanese and European vehicles are more fuel-efficient and more consistent than the historically heavier USA fleet.
- Average fuel efficiency rose across the observed decades, plausibly driven by the 1970s oil crises and the introduction of CAFE standards.
- A from-scratch Normal Equation regression matches `np.polyfit()` exactly, and PCA confirms that most of the feature variance collapses onto a single "car size" axis.